

第7章 排序

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握七种排序的算法思想与实现,理解**稳定性**并会给出反例
- 能用求和记号 Σ 推导插入排序的最坏 $O(n^2)$ 与"近有序 $O(n)$ "性质
- 掌握快排的**三数中值分割法**与小区间插入优化,说清最坏 $O(n^2)$ 何时发生
- 能用决策树直觉解释**比较下界** $\Omega(n \log n)$,理解桶排为何能突破
- 完成从 C 的 `qsort` 到 C++ `std::sort` (模板 + lambda)的迁移,能按数据特征选型

💡 从 C 到 C++:本章主题如何迁移

排序是工程中出现频率最高的操作:排行榜、去重、二分查找前置、Top-K.....本章把第2章的复杂度分析、第6章的二叉堆全部用上。路线:经典排序(\$7.1-\$7.4) → 快排(\$7.5) → 桶排与外部排序(\$7.6) → 总表选型(\$7.7)。请务必亲手写一遍每一种排序——它们最常被要求默写。

在 C 里,通用排序武器只有 `qsort`,三个痛点——① `void*` 丢类型;② 比较语义是"负/零/正"三值;③ 只能整体排序。C++ 的 `std::sort` 用模板 + 迭代器逐一解决:类型由模板推导,lambda 给出"是否小于"的布尔语义,迭代器区间任意截取;它内部是"快排 + 堆排 + 插入"混合体(introsort),本章三种优化全内置。

📦 前置知识自查

数学前置:求和记号 Σ 与 $\Sigma_{i=1}^n i = n(n+1)/2$;递推式 $T(n) = 2T(n/2) + n$ (第2章)。

C 语言前置:数组与 `for`;函数指针与 `void*` (\$7.5 对照)。

依赖章节:第2章复杂度;第6章二叉堆(堆排序复用**下滤**与**下滤建堆**,注意 1 基/0 基差异)。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 \$7.1-\$7.2,默写插入与希尔排序
2. **第二阶段(1.5 小时):**读 \$7.3-\$7.4,手写堆排与 merge
3. **第三阶段(1.5 小时):**读 \$7.5-\$7.6,吃透三数中值与分割
4. **第四阶段(实验,1 小时):**完成章末实验。验收:能默写全部经典排序,解释快排何时最坏

本章知识导图

- §7.1 插入排序:扑克牌直觉、 Σ 推导、近有序 $O(n)$ 、稳定性
- §7.2 希尔排序:希尔增量与 Hibbard、约 $O(n^{3/2})$ 、不稳定
- §7.3 堆排序:复用第6章的二叉堆(朴素版、原地版、ADT 关系)
- §7.4 归并排序: $T(n)=2T(n/2)+n$ 、merge、稳定与 $O(n)$ 空间
- §7.5 快速排序:三数中值、分割、小区间插入、qsort $\rightarrow$ std::sort 对照
- §7.6 桶式排序与外部排序:桶排 $O(n)$ 、比较下界 $\Omega(n \log n)$ 、多路归并
- §7.7 排序算法总表:七种排序大表、选型决策树

§7.1 插入排序:算法、复杂度与稳定性

7.1.1 扑克牌直觉与算法步骤

直觉一句话:像整理扑克牌——每摸一张新牌,插到已排好序的牌中的正确位置。算法把数组分成"已排序前缀 $a[0..i-1]$ "与"未排序后缀 $a[i..n-1]$ ":第 i 步用 key 暂存 $a[i]$,从右往左找位置,挡路元素右移一格,落位。

示例 7-1 插入排序(key 暂存与右移)

```
// 对 a[left..right] 升序排序
void insertionSort(vector<int>& a, int left, int right) {
    for (int i = left + 1; i <= right; ++i) {
        int key = a[i];
        int j = i - 1;
        while (j >= left && a[j] > key) {    // 严格大于才右移:稳定
            a[j + 1] = a[j];
            --j;
        }
        a[j + 1] = key;
    }
}
```

7.1.2 复杂度推导:把循环翻译成求和

最坏情形(完全逆序):第 i 个元素要与前面 i 个元素逐一比较并右移,总比较次数 $= \sum_{i=1}^{n-1} i = n(n-1)/2 = O(n^2)$ 。平均情形:第 i 个元素平均只比较 $i/2$ 次,总次数 $\approx n(n-1)/4$,仍 $O(n^2)$ 。最好情形(已有序):每个元素只比较 1 次, $O(n)$ 。

更本质的刻画是**逆序对(inversion)**:满足 $i < j$ 且 $a[i] > a[j]$ 的数对。插入排序每次右移恰好消除一个逆序对,比较/移动次数 = 逆序对数——逆序对很少时排序就快,"近有序 $O(n)$ "的来源。

7.1.3 近有序数组:接近 $O(n)$ 的性质

若数组"大体有序"(逆序对只有 $O(n)$ 个),插入排序只需 $O(n)$ 次比较与移动,接近线性——章末实验将实测。但也暴露致命局限:每次只交换相邻元素,跨越 k 个位置要走 k 步。例:[2, 3, 4, 5, 6, 7, 8, 1] 中元素 1 走到开头要走 7 步。"让元素一次走远"正是 §7.2 希尔排序的动机。

7.1.4 稳定性:相等元素保持相对次序

稳定性(stable):值相等的元素保持原相对次序。插入排序是稳定的——右移条件写"严格大于" `a[j] > key`,相等立即停止。为何重要?先按姓名排、再按分数排,若第二次稳定,同分者仍按姓名有序——多关键字排序可逐关键字轮流排。

⚠ 误区 1:内层循环里不暂存 key,直接与 `a[i]` 交换

新手常写 `while (a[j] > a[i]) { a[j+1] = a[j]; --j; } a[j+1] = a[i];` —— `a[i]` 在右移中可能已被覆盖,key 丢失。正确姿势:先 `int key = a[i]`,循环里只移动 `a[j]`。

§7.2 希尔排序:增量序列与远距离搬移

7.2.1 动机:让元素一次走远

插入排序慢在"每次只走一格"。希尔排序:先用大步长"预排序"让元素大体到位,再逐步缩小步长,步长 1 时就是普通插入排序。步长叫**增量(gap)**:把下标相差 `h` 的元素看作同一组,对每组做插入排序——元素一次跨越 `h` 个位置。

7.2.2 增量序列:希尔增量与 Hibbard

- **希尔增量:**`n/2, n/4, ..., 1`(每次折半)。实现最简单,但增量互为倍数,奇偶下标各自成组,最坏退化为 $O(n^2)$;
- **Hibbard 增量:**`1, 3, 7, 15, ..., (2^k - 1)`。相邻增量互质,避开坏结构,最坏 $O(n^{3/2})$ 。

示例 7-2 希尔排序(希尔增量版)

```
void shellSort(vector<int>& a) {
    int n = int(a.size());
    for (int gap = n / 2; gap > 0; gap /= 2) { // 希尔增量:n/2, n/4, ..., 1
        for (int i = gap; i < n; ++i) { // 对每组做插入排序
            int key = a[i];
            int j = i;
            while (j >= gap && a[j - gap] > key) { // 一次跳 gap 格
                a[j] = a[j - gap];
                j -= gap;
            }
            a[j] = key;
        }
    }
}
```

7.2.3 复杂度与稳定性

希尔排序的精确复杂度至今未完全解决。工程口径(与速查表一致):希尔增量最坏 $O(n^2)$,Hibbard 增量最坏 $O(n^{3/2})$,平均都约 $O(n^{3/2})$ 。它不稳定:大间隔插入把相等元素"隔空换位"。反例:`[5a, 5b, 1]`,`gap = 2` 时 `(5a, 1)` 排成 `(1, 5a)`,数组变 `[1, 5b, 5a]`,最后 `gap = 1` 也救不回来。

⚠ 误区 2:增量序列忘了最后是 1,或增量间互为倍数

① 增量必须以 1 收尾,否则只保证"每组内有序",整体未必有序——`gap` 写成 `gap > 1` 就错。② 希尔增量互为倍数,偶数下标元素永远只在偶数组内比较,是最坏 $O(n^2)$ 的根源;改用 Hibbard($2^k - 1$) 互质序列即可避开。

§7.3 堆排序:复用第6章的二叉堆

7.3.1 朴素版:建堆 + 连续删除最小

第6章结尾预告过:堆排序 = 二叉堆的"全量消费"。朴素版两步:① 下滤建堆 $O(n)$;② 连续 `deleteMin` n 次,每次 $O(\log n)$ 。总代价 $O(n \log n)$,但要一块 $O(n)$ 输出数组。

7.3.2 原地版:最大堆 + 交换 + 缩小

原地版把输出数组省掉:改用**最大堆**。建最大堆后根即全局最大;把根与末尾交换,堆大小减 1,再下滤新根——被换到末尾的最大元素就此"钉死"在最终位置。反复 $n-1$ 次,全程只用原数组。

注意下标习惯:原地堆排用 0 基 (左子 $2i+1$ 、右子 $2i+2$ 、父 $(i-1)/2$ 、最后非叶 $n/2-1$),与第6章手写堆的 1 基不同。

示例 7-3 原地堆排序(0 基下滤)

```
// 下滤 a[i]; n 为当前堆大小
void siftDown(vector<int>& a, int i, int n) {
    while (true) {
        int largest = i;
        for (int c : {2 * i + 1, 2 * i + 2}) // 左子、右子(0 基)
            if (c < n && a[c] > a[largest])
                largest = c;
        if (largest == i) break;
        swap(a[i], a[largest]);
        i = largest;
    }
}

void heapSort(vector<int>& a) {
    int n = int(a.size());
    for (int i = n / 2 - 1; i >= 0; --i) // 1. 下滤建堆 O(n)
        siftDown(a, i, n);
    for (int i = n - 1; i > 0; --i) { // 2. 逐个钉死最大元
        swap(a[0], a[i]);
        siftDown(a, 0, i);
    }
}
```

7.3.3 复杂度、稳定性与 ADT 关系

建堆 $O(n)$ + $n-1$ 次下滤 $\times O(\log n) = O(n \log n)$,最好最坏都是它,且**原地**只要 $O(1)$ 额外空间。它不稳定:反例 $[4_a, 4_b, 3]$ 建堆后根 4_a 与末尾 3 交换得 $[3, 4_b, 4_a]$, 4_b 最终排在 4_a 前。

从 ADT 视角看:优先队列承诺"每次取出最小元",堆排序等于把"取最小"执行 n 次,顺便得到完整有序序列。"边插入边取最值"用优先队列,"一次性排好序"用排序 (§7.7)。

C 的手写堆排序	C++ 的标准库
手写下滤、建堆、交换,约 30 行	<code>make_heap</code> + <code>sort_heap</code> 两行

⚠ 误区 3:堆排序把 1 基公式抄进 0 基代码

第6章手写堆用 1 基,原地堆排用 0 基。把 1 基的"左子 $2i$ 、父 $i/2$ "抄进来会访问错位置,堆序被悄悄破坏、输出"看起来差不多但不对"。对策:下标公式写进注释,写完用"验证堆序"断言跑一遍。

§7.4 归并排序:分治的教科书

7.4.1 分治三步与递推式

归并排序是分治思想的教科书案例:① 分割:数组从中间切成两半;② 递归:各自排序;③ 合并:两个有序半区线性合并成一个有序整区。递推式即第2章的 $T(n) = 2T(n/2) + n$,解得 $O(n \log n)$ 。

7.4.2 merge:两个有序序列的线性合并

合并直觉:像两队人按身高排队,每次让队首较矮者出列。 i 扫左半、 j 扫右半、 k 写 tmp ;某半耗尽,另一半剩余整体搬入。

示例 7-4 归并排序(辅助数组全程复用一份)

```
// 合并 a[l..m] 与 a[m+1..r],写回 a[l..r]
void merge(vector<int>& a, vector<int>& tmp, int l, int m, int r) {
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        if (a[j] < a[i]) tmp[k++] = a[j++];    // 右半更小才取右
        else tmp[k++] = a[i++];                // 相等取左 → 稳定
    }
    while (i <= m) tmp[k++] = a[i++];
    while (j <= r) tmp[k++] = a[j++];
    for (k = l; k <= r; ++k) a[k] = tmp[k];
}

void mergeSortRec(vector<int>& a, vector<int>& tmp, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSortRec(a, tmp, l, m);
    mergeSortRec(a, tmp, m + 1, r);
    merge(a, tmp, l, m, r);
}

void mergeSort(vector<int>& a) {
    vector<int> tmp(a.size());                // 唯一一份辅助数组,全程复用
    mergeSortRec(a, tmp, 0, int(a.size()) - 1);
}
```

🚀 实现建议:C++ 必须显式管理辅助数组

C++ 没有"列表拼接"语法,合并用一块全程复用的 tmp (示例 7-4):每次 merge 新建临时 vector 会每层递归反复分配,常数翻几倍。辅助数组只分配一次,在递归层间共享。

7.4.3 稳定性与空间

归并排序**稳定**:合并"相等取左半"(严格小于才取右)。代价是 $O(n)$ 额外空间。工程价值:Python 的 `sorted` 用的 Timsort 就是"归并 + 插入"混合;C++ 要稳定排序用 `std::stable_sort`。递归深度仅 $\log_2 n$,无栈溢出风险。

⚠ 误区 4:merge 的边界错误(漏掉剩余半区或忘记拷回)

两个高频错:① 主循环后忘了把某半剩余元素搬进 tmp(奇数长度必错);② 结果只写进 tmp、忘了拷回原数组(上层读到旧数据)。调试技巧:用 5 个元素手工走一遍。

§7.5 快速排序:工程之王

7.5.1 思想与最坏情形

快排也是分治,但与归并的"先递归后合并"相反:难点在"分割"——选一个**枢轴(pivot)**,把数组分成"小于枢轴"与"大于枢轴"两半,枢轴落到最终位置,再递归排序两半。分割后无需合并——这是它常数小的根源。平均枢轴居中,递推式仍是 $T(n) = 2T(n/2) + n, O(n \log n)$; 最坏 $O(n^2)$:每次分割极度不平衡——典型场景"已有序 + 首元素枢轴",退化为 $T(n) = T(n-1) + n$ 。

7.5.2 三数中值分割法

对付最坏情形的标准武器是**三数中值分割法(median-of-three)**:取左端、中间、右端三个元素,中值当枢轴,顺便把三者排好序——左、右端变成"哨兵",枢轴藏中间。已有序数组会选中真正的中间值,分割永远均衡,退化被根治。

示例 7-5 快速排序:三数中值 + 分割 + 小区间插入

```

const int CUTOFF = 16;    // 长度 ≤ 16 改用插入排序

int median3(vector<int>& a, int left, int right) {
    int center = (left + right) / 2;
    if (a[center] < a[left]) swap(a[left], a[center]);
    if (a[right] < a[left]) swap(a[left], a[right]);
    if (a[right] < a[center]) swap(a[center], a[right]);
    swap(a[center], a[right - 1]);    // 枢轴藏到 right-1
    return a[right - 1];
}

void quickSortRec(vector<int>& a, int left, int right) {
    if (left + CUTOFF <= right) {
        int pivot = median3(a, left, right);
        int i = left, j = right - 1;
        for (;;) {
            while (a[++i] < pivot) {} // 找 ≥ 枢轴者(a[left] 哨兵)
            while (pivot < a[--j]) {} // 找 ≤ 枢轴者(a[right-1] 哨兵)
            if (i < j) swap(a[i], a[j]);
            else break;
        }
        swap(a[i], a[right - 1]);    // 枢轴归位
        quickSortRec(a, left, i - 1);
        quickSortRec(a, i + 1, right);
    } else {
        insertionSort(a, left, right); // 小区间插入($7.1)
    }
}

```

为什么循环不会越界?三数中值已保证 $a[\text{left}] \leq \text{枢轴}$ 且 $a[\text{right}] \geq \text{枢轴}$,两个"哨兵"让 `++i` 与 `--j` 必然停下——这正是"顺便排好序"的深意。

7.5.3 小区间插入排序优化

快排递归到小区间时,插入排序反而更快:递归有开销、插入常数极小、缓存友好。工程实现普遍在长度 ≤ 16 时切换: `CUTOFF = 16 = "快排大区间 + 插入小区间"`。 `std::sort` 也这么做。

7.5.4 ★从 C 的 qsort 到 C++ 的 std::sort

C 的 qsort(函数指针 + void*)	C++ 的 std::sort(模板 + lambda)
qsort(a, n, sizeof(int), cmp):返回负/零/正	sort(a.begin(), a.end()):默认升序,一行
int cmp(const void*, const void*) 手动强转,无类型检查	模板推导类型,错误编译期报错
自定义顺序只能写死返回值规则	lambda: sort(v.begin(), v.end(), [](int x, int y) { return x > y; }) 降序

示例 7-6 std::sort 的四种用法

```
#include <algorithm>
vector<int> v = {5, 2, 8, 1, 9};
sort(v.begin(), v.end()); // 升序
sort(v.begin(), v.end(), greater<int>()); // 降序
sort(v.begin(), v.end(),
    [](int x, int y) { return x % 10 < y % 10; }); // 按个位数升序
vector<string> names = {"bob", "alice", "carl"};
sort(names.begin(), names.end(),
    [](const string& x, const string& y) { return x.size() < y.size(); });
// 按长度升序;要稳定必须用 stable_sort!
```

⚠ 误区 5:把 C 的三值比较语义搬进 lambda

qsort 返回"负/零/正", `std::sort` 的比较器返回 `bool`。写 `[](int x, int y) { return x - y; }` 不报错但含义全错:返回 0 或负值都是"真"! 比较器永远写 `return x < y;`。这是 C 迁移时最高频的 bug。

7.5.5 快排的稳定性

快排**不稳定**:分割的"交换"会把相等元素隔空换位。反例:[3, 5_a, 2, 5_b, 1],取 3 当枢轴:分割时 5_a 与 1 交换,越过 5_b,次序破坏。需要稳定:小数据用插入,大数据用 `std::stable_sort`。

§7.6 桶式排序与外部排序:跳出"比较"的框架

7.6.1 桶式排序:用空间换时间

前面五种排序都属于"比较排序"。**桶式排序(bucket sort)** 完全不做比较:把值直接当下标,数每个值出现几次,再按顺序倒出来——即**计数排序(counting sort)**(每桶一个值)。复杂度 $O(n + M)$, M 是值域; $M = O(n)$ 时即 $O(n)$ 。

示例 7-7 计数排序(前缀和 + 倒序,稳定)

```
// 值域 [0, M) 的非负整数;cnt 兼作位置账本
void countingSort(vector<int>& a, int M) {
    vector<int> cnt(M, 0), out(a.size());
    for (int x : a) ++cnt[x]; // 1. 计数
    for (int i = 1; i < M; ++i) cnt[i] += cnt[i - 1]; // 2. 前缀和
    for (int i = int(a.size()) - 1; i >= 0; --i) // 3. 倒序扫描
        out[--cnt[a[i]]] = a[i]; // 稳定:相等后到先取
    a = out;
}
```

C 的计数排序

```
int *cnt = calloc(M,
sizeof(int)); 用完 free,忘了即泄漏
```

C++ 的计数排序

```
vector<int> cnt(M, 0) 自动管理内存
```

7.6.2 比较下界: $\Omega(n \log n)$ 的决策树直觉

为什么桶排能是 $O(n)$?因为任何只靠"比较"的排序都有下界 $\Omega(n \log n)$,而桶排不比较。一句话直觉:一次比较最多把 $n!$ 种排列一分为二;要确定唯一正确排列,至少需要 $\log_2(n!) \approx n \log_2 n$ 次比较。 $O(n)$ 排序必须另辟蹊径——利用值域结构,而非元素间比较。

桶排的适用条件因此苛刻: ① 数据是(或可映射成)小范围整数;② 值域与数据量同阶;③ 分布大致均匀。工程上主要用于成绩统计、颜色直方图、基数排序底层($O(d(n+M))$)。

7.6.3 外部排序:内存装不下时

外部排序(external sort) 解决"数据比内存大"的问题(数据库、日志)。思路是归并的外延:数据切成若干块,每块内存排序后写回磁盘;再开小顶堆,放每个已排块的当前最小元素,每次弹出全局最小者写入输出,再从该块补进下一个——弹出、补入、再弹出直至读完;输出仍装不下就多趟合并。分块排序用 §7.4 的归并,"多路取最小"正是第6章的小顶堆——外部排序 = 归并思想 + 堆结构 + 磁盘代替内存。

⚠ 误区 6:对不满足条件的数据用桶排

对 10^6 个浮点数或值域 10^9 的整数用计数排序,内存直接爆炸(要 M 个计数器)。判断口诀: 值域多大?是整数吗?分布均匀吗?任何一条不满足,老老实实用 $O(n \log n)$ 通用排序。工程上 99% 的场景都是通用排序,桶排是值域受限场景的特供品。

§7.7 排序算法总表:选型决策

7.7.1 七种排序全景表

算法	平均	最坏	空间	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	约 $O(n^{3/2})$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
桶排(计数)	$O(n + M)$	$O(n + M)$	$O(M)$	稳定

读表要点: 只有插入、冒泡能利用已有序(最好 $O(n)$);只有归并和桶排稳定;只有堆排最坏 $O(n \log n)$ 且 $O(1)$ 空间;快排平均最快但最坏 $O(n^2)$ 。没有一种全占——选型即权衡。

7.7.2 选型决策树

- 数据量很小或基本有序 → **插入排序**
- 需要稳定、空间允许 → **归并排序** (`std::stable_sort`)
- 内存紧张、最坏必须保证 → **堆排序**
- 通用场景、要快 → **快速排序**
- 值域小的整数 → **桶排/计数排序**
- 其余一切情况 → **标准库 sort/sorted**,不要手写

7.7.3 标准库是工程默认

最后一条最重要: 工程上永远优先 `std::sort` ——它是 introsort:快排为主,递归过深自动切堆排(防最坏退化),小区间自动切插入排序。本章手写的三种优化标准库全部内置,还多一道防退化保险。`std::stable_sort` 走归并家族,稳定且同样快。手写排序的意义:理解原理、应对考试;生产代码直接用标准库——本章实验"手写三种排序并统计比较次数",正是为亲手验证第2章的复杂度理论。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
插入排序	随机/逆序数组	$O(n^2)$	$O(n^2)$
插入排序	已有序数组	$O(n)$	$O(n)$
冒泡排序	相邻交换	$O(n^2)$	$O(n^2)$
希尔排序	希尔增量	约 $O(n^{3/2})$	$O(n^2)$
希尔排序	Hibbard 增量	约 $O(n^{3/2})$	$O(n^{3/2})$
堆排序	建堆阶段	$O(n)$	$O(n)$
堆排序	完整排序	$O(n \log n)$	$O(n \log n)$
归并排序	完整排序	$O(n \log n)$	$O(n \log n)$
归并排序	额外空间	$O(n)$	$O(n)$
快速排序	随机数据	$O(n \log n)$	$O(n \log n)$
快速排序	已有序 + 首元素枢轴	$O(n^2)$	$O(n^2)$
桶排(计数排序)	值域 M	$O(n + M)$	$O(n + M)$
比较下界	基于比较的排序	$\Omega(n \log n)$ (决策树)	
归并两个有序数组	线性合并	$O(n)$	$O(n)$
外部排序	多路归并	磁盘 I/O 主导;比较部分 $O(n \log n)$	

最小必会清单

- 能默写插入排序(key 暂存三步),用 Σ 推导最坏 $O(n^2)$ 、近有序 $O(n)$
- 能说出希尔排序动机、希尔增量与 Hibbard 差别、复杂度约 $O(n^{3/2})$
- 能默写原地堆排序(0 基下滤、建堆、交换缩小),说出它不稳定并给反例
- 能默写归并的 merge,说清"相等取左"保证稳定、辅助数组 $O(n)$ 全程复用
- 能默写快排的三数中值 + 分割 + 小区间插入,解释哨兵防越界
- 能说出快排最坏 $O(n^2)$ 何时发生、三数中值为何能治
- 能说出比较下界 $\Omega(n \log n)$ 的决策树直觉与桶排为何突破
- 能说出桶排三个适用条件,写出稳定版计数排序
- 能背出七种排序复杂度总表(最好/平均/最坏/空间/稳定性)

- 能写 `std::sort` 的 lambda 比较器与 `stable_sort`,说出与 `qsort` 三值语义的区别

自测题

Q 1. 插入排序对已有序数组的复杂度是多少?为什么?

✓ 参考答案

$O(n)$ 。每个元素与前驱比较 1 次就停止,共 $n-1$ 次比较、0 次移动。这也是"近有序接近 $O(n)$ "的来源:总代价 = 逆序对数。

Q 2. 堆排序为什么不稳定?构造一个反例。

✓ 参考答案

建堆与"根与末尾交换"都可能让相等元素换位。反例:[$4_a, 4_b, 3$] 建最大堆后根 4_a 与末尾 3 交换得 [$3, 4_b, 4_a$], 4_b 最终在前。

Q 3. 归并排序的空间复杂度是多少?为什么它不能像堆排那样原地?

✓ 参考答案

$O(n)$ 辅助数组。合并时两半在原数组中交错,原地交换会破坏"两半各自有序"的前提;原地归并要么 $O(n^2)$ 时间,要么复杂块交换技巧,得不偿失。

Q 4. 快排的最坏情形是什么?三数中值分割法为什么能缓解?

✓ 参考答案

每次分割都极度不平衡时(递推式 $T(n) = T(n-1) + n \rightarrow O(n^2)$),典型场景是已有序数组配"首元素枢轴"。三数中值取左、中、右的中值:已有序数组的中值恰是中间值,分割均衡;顺带排好 $a[\text{left}] \leq \text{枢轴} \leq a[\text{right}]$ 作哨兵。

Q 5. 为什么桶排能做到 $O(n)$,却不受"比较下界 $\Omega(n \log n)$ "的限制?

✓ 参考答案

决策树论证只适用于"通过比较获得信息"的算法:每次比较最多把 $n!$ 种排列一分为二,故至少 $\log_2(n!) \approx n \log_2 n$ 次。桶排不比较,直接用值域做索引(计数),绕开决策树,所以可 $O(n)$;代价是空间 $O(M)$ 与苛刻的适用条件。

章末实验题:三种排序的性能对比实验

实验 7:三种排序性能对比(三种数据分布)

实验目标:实现插入、归并、快排,在随机/近有序/逆序三种分布上实测 $n = 1000 \dots 64000$ 的耗时,用" n 翻倍时间比值"验证理论复杂度。

实验要求:

- 任务 1:实现 insertionSort、mergeSort、quickSort(\$7.1/\$7.4/\$7.5)
- 任务 2:三种分布 $\times$ 七个规模 n 的耗时表;插入 $n > 8000$ 标注"(推算)"
- 任务 3:与 `std::sort` 对比验证输出一致,含重复元素
- 任务 4:" n 翻倍时间比值"与理论对比:插入 $\times 4(n^2)$ 、归并/快排 $\approx \times 2(n \log n)$
- 任务 5(加分):统计比较/交换次数,验证插入比较次数 $\approx n^2/4$ (逆序对)

实验环境:C++17。编译: `g++ -std=c++17 -O2 lab7.cpp -o lab7`;运行: `./lab7`。

第一步 排务必用三数中值 + CUTOFF=16 小区间插入(\$7.5 完整版),否则逆序数据退化 $O(n^2)$,与理论演示矛盾

第二步 时取"同数据 3 次最小值"、用副本、结果读走防优化,固定种子 12345 可复现;近有序用"已排序 + 固定 20 次随机交换"生成(逆序对 $O(n)$),才能观察插入的 $O(n)$ 特性

第三步 务 4 比值只看"翻倍相邻行",小 n 计时噪声大,偏离属正常,看趋势即可

✅ 参考答案(完整可运行程序,分六段)

```
// lab7.cpp — 三种排序的性能对比实验(插入/归并/快排)
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>
using namespace std;
using Clock = chrono::steady_clock;

const int CUTOFF = 16;          // 快排小区间阈值($7.5)
long long g_cmp = 0, g_move = 0;
bool g_count = false;          // 任务 5 计数开关(计时时关)
inline bool lt(int x, int y) { if (g_count) ++g_cmp; return x < y; }
inline void exch(int& x, int& y) { if (g_count) ++g_move; swap(x, y); }

void insertionSort(vector<int>& a, int left, int right) {
    for (int i = left + 1; i <= right; ++i) {
        int key = a[i];
        int j = i - 1;
        while (j >= left && lt(key, a[j])) { // 严格大于才右移:稳定
            a[j + 1] = a[j];
            if (g_count) ++g_move;
            --j;
        }
        a[j + 1] = key;
        if (g_count) ++g_move;
    }
}

void insertionSortAll(vector<int>& a) { insertionSort(a, 0, int(a.size()) - 1); }
```

lab7.cpp(第二段:归并排序)

```

void merge(vector<int>& a, vector<int>& tmp, int l, int m, int r) {
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        if (lt(a[j], a[i])) tmp[k++] = a[j++]; // 右半更小才取右:相等取左,稳定
        else tmp[k++] = a[i++];
        if (g_count) ++g_move;
    }
    while (i <= m) { tmp[k++] = a[i++]; if (g_count) ++g_move; }
    while (j <= r) { tmp[k++] = a[j++]; if (g_count) ++g_move; }
    for (k = l; k <= r; ++k) { a[k] = tmp[k]; if (g_count) ++g_move; }
}

void mergeSortRec(vector<int>& a, vector<int>& tmp, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSortRec(a, tmp, l, m);
    mergeSortRec(a, tmp, m + 1, r);
    merge(a, tmp, l, m, r);
}

void mergeSort(vector<int>& a) {
    vector<int> tmp(a.size()); // 辅助数组只分配一次
    mergeSortRec(a, tmp, 0, int(a.size()) - 1);
}

```

lab7.cpp(第三段:快速排序与数据生成)

```

int median3(vector<int>& a, int left, int right) {
    int center = (left + right) / 2;
    if (lt(a[center], a[left])) exch(a[left], a[center]);
    if (lt(a[right], a[left])) exch(a[left], a[right]);
    if (lt(a[right], a[center])) exch(a[center], a[right]);
    exch(a[center], a[right - 1]); // 枢轴藏到 right-1
    return a[right - 1];
}

void quickSortRec(vector<int>& a, int left, int right) {
    if (left + CUTOFF <= right) {
        int pivot = median3(a, left, right);
        int i = left, j = right - 1;
        for (;;) {
            while (lt(a[++i], pivot)) {} // 哨兵 a[left]<=pivot 保证能停
            while (lt(pivot, a[--j])) {} // a[right-1]==pivot 保证能停
            if (i < j) exch(a[i], a[j]);
            else break;
        }
        exch(a[i], a[right - 1]); // 枢轴归位
        quickSortRec(a, left, i - 1);
        quickSortRec(a, i + 1, right);
    } else {
        insertionSort(a, left, right); // 小区间优化($7.1 预告)
    }
}

void quickSort(vector<int>& a) { quickSortRec(a, 0, int(a.size()) - 1); }

enum Kind { RANDOM, NEARLY, REVERSED };
const char* kindName[] = {"随机", "近有序", "逆序"};
vector<int> makeData(int n, Kind k, mt19937& rng) {
    vector<int> a(n);
    for (int i = 0; i < n; ++i) a[i] = i + 1;
    if (k == RANDOM) {
        for (int i = n - 1; i > 0; --i)
            swap(a[i], a[uniform_int_distribution<int>(0, i)(rng)]);
    } else if (k == NEARLY) {
        for (int i = 0; i < 20; ++i) // 固定 20 次随机交换:逆序对 O(n)
            swap(a[uniform_int_distribution<int>(0, n - 1)(rng)],
                a[uniform_int_distribution<int>(0, n - 1)(rng)]);
    } else {
        for (int i = 0; i < n; ++i) a[i] = n - i;
    }
    return a;
}

```

| lab7.cpp(第四段:计时与正确性验证)

```

// 计时:3 次取最小,每次用副本,结果读走防优化
double msTime(void (*f)(vector<int>&), vector<int> a, long long& guard) {
    double best = 1e18;
    for (int r = 0; r < 3; ++r) {
        vector<int> b = a;
        auto t0 = Clock::now();
        f(b);
        guard += b.front();
        best = min(best, chrono::duration<double, milli>(Clock::now() - t0).count());
    }
    return best;
}

int main() {
    mt19937 rng(12345);          // 固定种子,可复现
    const vector<int> sizes = {1000, 2000, 4000, 8000, 16000, 32000, 64000};
    double ins[3][7] = {}, mrg[3][7] = {}, qck[3][7] = {};
    long long guard = 0;

    // 任务 3:正确性验证(与 std::sort 一致,含重复元素)
    bool ok = true;
    for (int k = 0; k < 4; ++k) {
        vector<int> a = makeData(500, Kind(k % 3), rng);
        if (k == 3) for (int& x : a) x = uniform_int_distribution<int>(0, 20)(rng);
        vector<int> ref = a, b;
        sort(ref.begin(), ref.end());
        b = a; insertionSort(b, 0, int(b.size()) - 1); ok &= (b == ref);
        b = a; mergeSort(b);                          ok &= (b == ref);
        b = a; quickSort(b);                          ok &= (b == ref);
    }
    cout << "任务 3:三种算法与 std::sort 输出一致: " << (ok ? "是" : "否") << "\n";
}

```

lab7.cpp(第五段:耗时表与比值分析)


```

// 任务 2:耗时表(插入 n>8000 标注"(推算)")
cout << "任务 2:耗时表(单位 ms;插入排序 n>8000 为(推算))\n";
for (int k = 0; k < 3; ++k) {
    cout << "\n[" << kindName[k] << "]\n";
    cout << left << setw(8) << "n" << setw(11) << "插入" << setw(11) << "归并" << "快排\n";
    for (size_t s = 0; s < sizes.size(); ++s) {
        vector<int> a = makeData(sizes[s], Kind(k), rng);
        cout << setw(8) << sizes[s];
        if (sizes[s] <= 8000) {
            ins[k][s] = msTime(insertionSortAll, a, guard);
            cout << setw(11) << fixed << setprecision(2) << ins[k][s];
        } else { ins[k][s] = -1; cout << setw(11) << "(推算)"; }
        mrg[k][s] = msTime(mergeSort, a, guard);
        qck[k][s] = msTime(quickSort, a, guard);
        cout << setw(11) << mrg[k][s] << qck[k][s] << "\n";
    }
}

// 任务 4:比值分析(理论:插入 x4;归并/快排 ≈x2)
cout << "\n任务 4:n 翻倍时实测时间比值(理论:n² 算法 x4,n log n 算法 ≈x2)\n";
auto ratios = [&](const char* name, double t[3][7]) {
    cout << name << ": ";
    for (int k = 0; k < 3; ++k) {
        cout << kindName[k] << "[";
        for (size_t s = 1; s < sizes.size(); ++s)
            if (t[k][s] > 0 && t[k][s - 1] > 0) cout << setprecision(2) << (t[k][s] / t[k][s - 1])
<< " ";
        cout << "]" ";
    }
    cout << "\n";
};
ratios("插入", ins);
ratios("归并", mrg);
ratios("快排", qck);

```

lab7.cpp(第六段:统计与结论)

```

// 任务 5(加分):比较/移动次数统计
g_count = true;
vector<int> a = makeData(1000, RANDOM, rng), b;
g_cmp = 0; g_move = 0; b = a; insertionSort(b, 0, int(b.size()) - 1);
cout << "\n任务 5(n=1000 随机):插入 比较=" << g_cmp
<< " 移动=" << g_move << "\n";
g_cmp = 0; g_move = 0; b = a; mergeSort(b);
cout << "          归并 比较=" << g_cmp << " 移动=" << g_move << "\n";
g_cmp = 0; g_move = 0; b = a; quickSort(b);
cout << "          快排 比较=" << g_cmp << " 交换=" << g_move << "\n";
g_count = false;

cout << "\n结论:实测比值与理论吻合—插入 n 翻倍约 x4,归并/快排约 x2;"
      "逆序上插入退化最明显,近有序上插入接近 O(n),快排三数中值使逆序不再退化。 \n";
cout << "(防优化累加值: " << guard << ")\n";
return 0;
}

```

示例输出(本机真实运行,不同机器数值不同,比值规律一致):

任务 3:三种算法与 `std::sort` 输出一致: 是

任务 2:耗时表(单位 `ms`;插入排序 $n > 8000$ 为(推算))

[随机]

n	插入	归并	快排
1000	0.06	0.03	0.02
2000	0.24	0.08	0.06
4000	0.93	0.20	0.13
8000	3.64	0.41	0.29
16000	(推算)	0.87	0.62
32000	(推算)	1.88	1.35
64000	(推算)	4.04	2.89

[近有序] 表与 [逆序] 表见完整输出:插入在近有序上快一个数量级(8000 时仅 `0.03ms`),逆序上最慢(`7.25ms`)

任务 4:n 翻倍时实测时间比值(理论: n^2 算法 $\times 4$, $n \log n$ 算法 $\approx \times 2$)

插入: 随机[3.77 3.91 3.91] 近有序[1.63 1.38 2.30] 逆序[3.87 3.94 3.99]

归并: 随机[2.53 2.46 2.07 2.14 2.17 2.14](近有序/逆序各列也均 ≈ 2)

快排: 随机[2.54 2.19 2.19 2.16 2.17 2.14](同上)

任务 5($n=1000$ 随机):插入 比较=251264 移动=251270

归并 比较=8720 移动=19952

快排 比较=10671 交换=4969

读表要点:① 插入在随机/逆序上比值 $\approx \times 4$ (吻合 n^2),近有序上 $\approx \times 2$ (吻合 $O(n)$);② 归并/快排 $\approx \times 2$ (吻合 $n \log n$, 三种分布皆然);③ 快排逆序不退化——三数中值选中中间值,§7.5 优化的实战验证;④ 插入比较 $251264 \approx 1000^2/4$,落在平均情形的理论上。

设计说明:① `lt / exch` 计数钩子使比较/移动定义与正文一致;② 计时取 3 次最小值、用新副本、结果读走防优化;③ 插入 $n > 8000$ 标"(推算)": $\times 4$ 规律已由 $1000 \rightarrow 8000$ 确认,与第2章实验一脉相承;④ 近有序用固定 20 次交换生成(逆序对 $O(n)$),固定种子 12345 可复现。

下一章预告:第8章 不相交集——`union/find` 与路径压缩,一种"只回答是/否连通"的轻量数据结构;第9章图论算法中的 Kruskal 最小生成树将直接使用它。